NexusOS

A multi-tenant business platform where authentication, authorization and tenant isolation were built as infrastructure — before any product module was allowed to depend on them.

Overview

NexusOS is a web platform designed to host multiple business capabilities — CRM, projects, finance, HR, inventory — behind one application shell, one identity model and one tenant boundary. Rather than shipping a module first and retrofitting multi-tenancy later, the identity, permission and isolation layers were solved first so that everything built on top inherits them.

The platform layer is complete and tested end to end. The product modules above it are, with the dashboard interface as the exception, not yet built.

My Role

Sole engineer — system architecture, backend, frontend, database design, security engineering, design system, test suite and CI pipeline.

Key Engineering Highlights

Tenant scope is derived, never accepted. Authenticated requests carry no tenant header, path segment or body field. Company identity is read from the caller's stored record on every request, so a valid token for one tenant cannot address another.

Authorization is re-derived from the database, not read from token claims. Role and permissions are loaded fresh on every protected request and fail closed on suspended users or deactivated tenants — so revocation takes effect on the next request, not at token expiry.

Refresh-token reuse is treated as compromise. Refresh tokens rotate against server-side session records; replaying a spent token revokes every session for that user and writes an audit event.

Isolation is enforced twice. Application-level scoping plus composite foreign keys — `(id, company_id)` — that make a cross-tenant relationship unrepresentable in the database, independent of application correctness.

Layered request security. CORS allowlisting with production startup validation, security headers, Redis-backed rate limiting on credential endpoints, and a trusted-proxy boundary so IP-based controls cannot be spoofed by a forwarded header.

A design system built as infrastructure. 157 design tokens and 31 components with a living style-guide route, plus a single-flight token refresh in the API client so concurrent 401s don't invalidate each other through rotation.

Technology

Frontend — React 19, TypeScript, Vite, React Router, TanStack Query, Zustand, React Hook Form, Zod, Tailwind CSS, Radix UI, Recharts

Backend — Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2.0 async, Alembic, Argon2, PyJWT, structlog

Data — PostgreSQL 16 via asyncpg, Redis 7 for rate-limit counters

Infrastructure & testing — Docker Compose, GitHub Actions CI, pytest against live PostgreSQL and Redis, Vitest with Testing Library, ruff, mypy

---

Status

Implemented — multi-tenant identity model, full authentication lifecycle, RBAC with database re-validation, the security middleware layer, and the frontend platform including the design system. 167 backend and 89 frontend test functions, with CI running lint, type checks, migrations and both suites.

Prepared — 38 domain model classes migrated for CRM, projects, finance, HR and inventory with no endpoints yet; a tenant-scoped repository implemented and tested but not yet on live routes; the dashboard interface complete against fixture data.

Not implemented — the product modules behind the sidebar, AI features (tables modelled, no inference code), and PostgreSQL row-level security. Tenant isolation is currently application-level and relational.